

Prediction task - House Prices

Completed as part of the coursework for the Data Mining and Knowledge Discovery in Data module at the University of Brighton, 2025/26.

Date: 11/06/2026

Table of Contents

Table of Contents	2
List of Figures.....	3
List of Tables	3
List of Appendices.....	3
1 Introduction	4
1.1 Background	4
1.2 Objective	4
2 Methodology	5
2.1 Data understanding.....	5
2.2 Preprocessing.....	7
2.3 Machine Learning Methods.....	9
2.4 Feature Selection	9
2.5 Model Building & Optimisation.....	12
3 Results	12
3.1 Evaluation	12
3.2 Comparison.....	14
3.3 Validation	15
4 Discussion	18
5 Conclusions.....	18
6 Ethics	19
References	20
Appendices	22
Appendix 2 – Elbow method	22
Appendix 1 – Geographical clusters	23
Appendix 3 – GridSearchCV-selected parameters	24

List of Figures

Figure 1. Bar chart of class balance assessment for binary features.	6
Figure 2. Bar chart of feature selection performance analyses.	11
Figure 3. Bar chart of RMSE percentage change after hyperparameter tuning.....	13
Figure 4. Bar chart of comparative model performance after hyperparameter tuning.....	14
Figure 5. Line plot of 5-fold cross-validation RMSE across models.....	15
Figure 6. Scatter plot of observed vs predicted values, Gradient Boost.	16
Figure 7. Histogram of residuals, Gradient Boost.	17
Figure 8. Bar chart of feature importance, Gradient Boost.	17

List of Tables

Table 1. House prices dataset original features.	5
Table 2. Data cleaning summary.	6
Table 3. House prices dataset features after preprocessing.	8
Table 4. Summary of results of feature selection methods.	10
Table 5. Comparative model performance, default hyperparameters.....	12
Table 6. GridSearchCV-selected hyperparameters (non-default).....	13
Table 7. Comparative model performance, tuned hyperparameters.	13

List of Appendices

Appendix 1. Elbow method for geographical clusters.....	22
Appendix 2. Map plot of geographical clusters.	23
Appendix 3. GridSearchCV-selected parameters.....	26
Appendix 4. Full code and output.	Error! Bookmark not defined.

1 Introduction

1.1 Background

Accurate prediction of residential property prices is of considerable economic importance, informing decisions made by buyers, sellers, investors, and policymakers. Predicting house prices is challenging — property values are influenced by a complex interplay of intrinsic and extrinsic factors with non-linear relationships that limit the effectiveness of traditional statistical approaches.

Machine learning methods have emerged as a powerful alternative, capable of capturing complex patterns in large datasets. Regression-based, Ensemble, Neural Networks, and Stacked approaches have all been applied to house price prediction with promising results (Madhuri, Anuradha and Pujitha, 2019; Aljohani, 2022; Chenxi, 2024; Sidhu *et al.*, 2025).

1.2 Objective

The primary objective of this study is to identify the best-performing machine learning model for predicting residential property sale prices using the King County dataset. Five candidate models of varying complexity are evaluated: Multiple Linear Regression, Random Forest, Gradient Boosting, a Stacked Ensemble, and a Multi-layer Perceptron.

A secondary objective is to assess the impact of hyperparameter tuning on model performance. Specifically, whether tuning yields meaningful improvements, and whether the relative ranking of models remains consistent before and after optimisation, building on evidence that tuning can significantly enhance predictive performance in the house price prediction task (Sharma, Harsora and Ogunleye, 2024).

To achieve these objectives, feature selection methods was applied to identify the optimal set of predictors, and 5-fold cross-validation was used to assess model stability and prevent overfitting. Together, these objectives aimed to provide insight into both the choice of modelling approach and the value of hyperparameter optimisation for house price prediction.

2 Methodology

2.1 Data understanding

2.1.1 Data overview

The dataset comprises residential property transactions with 21 original variables and 21,614 observations. The target variable is house price (\$, int64). The 20 predictor variables (features) –Table 1– include intrinsic factors such as structural characteristics, location, size-related variables, and qualitative attributes of the property, as well as the sale date and the year the house was built and, if applicable, renovated.

For the analyses, the ordinal features such as view, condition, and grade were treated as continuous numeric variables. Whilst this is appropriate for tree-based models, Linear Regression and the Neural Network implicitly assume equal spacing between ordinal levels, which may not hold in practice and represents a limitation of the current approach.

Table 1. House prices dataset original features.

Feature	Type	Description	Feature	Type	Description
id	int64	identifier	grade	int64	house grade (1-13)
date	str	sales date	sqft_above	int64	excl.basement
bedrooms	int64	count	sqft_basement	int64	basement only
bathrooms	float64	count	yr_built	int64	year house built
sqft_living	int64	sq footage house	yr_renovated	int64	year renovated
sqft_lot	int64	sq footage lot	zipcode	int64	zip code
floors	float64	count	lat	float64	latitude
waterfront	int64	True/False	long	float64	longitute
view	int64	quality of view (0-4)	sqft_living15	float64	sq footage 15 *
condition	int64	house cond. (1-5)	sqft_lot15	float64	sqft lot 15 *

* Sqft_living15 is variably defined in the literature — described by some sources as the square footage of interior living space of the nearest 15 neighbours (Chenxi, 2024), and by others as the living area measured in 2015, implying possible renovations (course materials).

2.1.2 Data cleaning

Initial data exploration identified an erroneous value (33 bedrooms) and four missing sqft values (NaN). The corrections were applied, as shown in Table 2.

Table 2. Data cleaning summary.

Observation id	Variable	Error value	Correction
2402100895	bedrooms	33	3
16000397	sqft_lot15	NaN	median
1873100390	sqft_living15	NaN	median
2426039314	sqft_lot15	NaN	median
9435300030	sqft_living15	NaN	median

2.1.3 Class balance assessment

Assessment of binary features in Figure 1 revealed varying degrees of class imbalance — waterfront was the most severely imbalanced, with fewer than 1% of properties, followed by renovation (4%) and basement (39%). Since this is a regression task, class imbalance in a predictor does not carry the same implications as in classification, and no resampling was applied. All three features were retained because minority classes — particularly waterfront and renovated properties — are likely to convey disproportionate price signals despite their rarity.

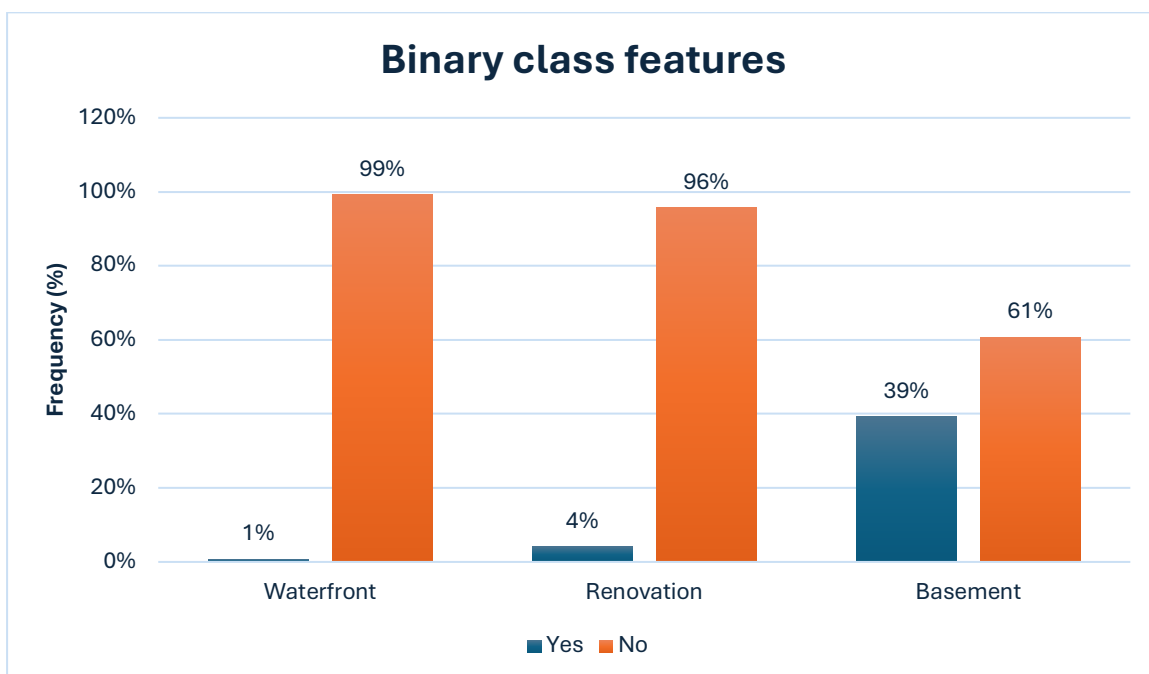


Figure 1. Bar chart of class balance assessment for binary features.

2.2 Preprocessing

Data preprocessing included several steps: feature engineering, log transformations, feature removal, train-test split, and scaling.

2.2.1 Feature engineering

The `sales_month` variable was extracted from the transaction date to capture potential seasonal patterns in house prices, as demand in property markets is known to fluctuate across the year. Two binary indicator variables were derived to simplify sparse continuous fields: `has_basement` (from `sqft_basement`, where any non-zero value was coded as 1) and `is_renovated` (from `yr_renovated`, indicating whether a property had ever undergone renovation). Additionally, `house_age` was calculated as the difference in years between the transaction date and `yr_built`, reflecting the intuition that older properties may command different prices due to condition or character. For renovated properties only, `years_since_renovation` was derived to capture how recently improvements were made, on the basis that the recency of renovation is likely to influence buyer perception and therefore price.

Longitude and latitude were also considered candidates for spatial feature engineering, with k-means clustering explored to group properties by geographic location (Appendix 1 and Appendix 2). However, clustering did not yield a meaningful improvement in model performance, and the added complexity was not considered justified. As a result, `long` and `lat` were retained in their original form, as continuous predictors.

2.2.2 Log transformations

Several area-based square footage features exhibited notable right skew and were therefore log-transformed to reduce the influence of extreme values and better approximate a normal distribution. Where features contained zero values, `log1p` was applied to avoid undefined outputs; otherwise, the standard log transformation was used. The target variable, `price`, was also log-transformed, so model predictions were generated on the log scale and then exponentiated to obtain interpretable price estimates.

2.2.3 Redundant feature removal

Following feature engineering and transformation, a number of variables were removed from the dataset to avoid redundancy. This included identifiers with no predictive value (`id`), categorical fields unsuitable for direct numerical modelling (`zipcode`), and variables used solely as inputs to derived features — namely `date`, `yr_built`, and `yr_renovated` — which were dropped once their engineered counterparts had been created. Square footage features that had been log-transformed, such as `sqft_living`, `sqft_lot`, and `sqft_above`, were also removed in favour of their transformed equivalents to prevent duplication.

2.2.4 Train-Test split

The dataset was split 80/20 into training and test sets, with 80% of observations used to train each model and the remaining 20% held back for evaluation. This split was applied consistently across all models to ensure that performance metrics remained comparable.

2.2.5 Scaling

StandardScaler was applied to selected continuous features and to those that followed the natural order before feature selection, transforming them to have a mean of zero and a standard deviation of one, ensuring that the selection process was not influenced by differences in variable magnitudes. Scaling is directly relevant to the Linear Regression and Neural Network models, where the scale of input features affects coefficient estimation and gradient updates, respectively. Whilst it has no bearing on the behaviour of Random Forest and Gradient Boosting — both of which are tree-based models that learn through threshold-based splits and are therefore invariant to feature scale — it was applied consistently across all models to maintain a uniform preprocessing pipeline and should not adversely affect their performance.

2.2.6 Summary

Preprocessing introduced five derived features — sales_month, has_basement, is_renovated, house_age, and years_since_renovation — and removed five redundant features — id, date, yr_built, yr_renovated, and zipcode. Selected numeric features were log-transformed where appropriate, and continuous features were standardised using StandardScaler. The resulting 20-feature matrix, Table 3, was subsequently passed to the feature selection procedure for further dimensionality reduction prior to modelling.

Table 3. House prices dataset features after preprocessing.

Feature	Type	Feature	Type
bedrooms	int64	is_renovated	int64
bathrooms	float64	years_since_renovation	int64
floors	float64	has_basement	int64
waterfront	int64	house_age	int64
view	int64	log_sqft_living	float64
condition	int64	log_sqft_lot	float64
grade	int64	log_sqft_above	float64
lat	float64	log_sqft_living15	float64
long	float64	log_sqft_lot15	float64

2.3 Machine Learning Methods

2.3.1 Models overview

Five candidate models were implemented using the scikit-learn library in Python and were considered for this analysis to represent a range of modelling approaches.

LinearRegression

Linear Regression served as a simple, interpretable baseline model that estimates the linear relationship between input features and a continuous target variable by minimising the residual sum of squares (scikit-learn, 2026c).

RandomForestRegressor

Random Forest is an ensemble learning method that constructs multiple decision trees on random subsets of the data and aggregates their predictions, capturing non-linear relationships whilst reducing overfitting through averaging (scikit-learn, 2026e).

GradientBoostingRegressor

Gradient Boosting is also tree-based but uses a sequential approach, in which each tree corrects the errors of the previous one, making it particularly effective at capturing complex feature interactions (scikit-learn, 2026a).

StackingRegressor

Stacking Regressor is an ensemble method that combines predictions from multiple base models via a meta-model to improve predictive performance by leveraging model diversity (scikit-learn, 2026g).

MLPRegressor

A multi-layer perceptron is a neural network-based model that uses multiple layers of interconnected nodes to capture complex, non-linear relationships between input features and the target variable (scikit-learn, 2026d).

2.4 Feature Selection

2.4.1 Procedures

Following preprocessing, selection procedures were applied across filter, wrapper, and embedded methods (scikit-learn, 2026f).

The Variance Threshold identified features with variance below 0.01 (near-constant) for removal; however, only the waterfront feature met this criterion, likely reflecting class imbalance in the dataset. Correlation identified and removed features that were highly correlated with another feature (above a threshold of 0.8), reducing multicollinearity and retaining only one representative feature from each correlated pair. ANOVA

selected the top 10 features based on the F-statistic from a regression test, which measured the linear relationship between each feature and the target variable, price. Mutual Information selected the top 10 features based on the statistical dependency between each feature and the target, capturing both linear and non-linear relationships, making it a more flexible alternative to ANOVA. Lasso fitted a LassoCV model with cross-validation and retained only features with non-zero coefficients, exploiting Lasso's built-in tendency to shrink irrelevant feature coefficients to exactly zero. Ridge fitted a RidgeCV model and retained features whose absolute coefficient exceeded 0.01, using coefficient magnitude as a proxy for feature relevance, though unlike Lasso, Ridge does not produce exact zero coefficients. Random Forest ranked features by their importance scores from a fitted Random Forest model and retained the top 10, capturing non-linear relationships and feature interactions that linear methods may overlook. Results are summarised in Table 4.

2.4.2 Selection criteria

Table 4. Summary of results of feature selection methods.

Feature	Variance Thresh.	Correlation	ANOVA	Mutual Inform.	Lasso (L1 reg.)	Ridge (L2 reg.)	Random Forest	Selected by
grade	✓	✓	✓	✓	✓	✓	✓	7
lat	✓	✓	✓	✓	✓	✓	✓	7
log_sqft_living	✓	✓	✓	✓	✓	✓	✓	7
log_sqft_living15	✓	✓	✓	✓	✓	✓	✓	7
bedrooms	✓	✓	✓	✓	✓	✓	✗	6
bathrooms	✓	✓	✓	✓	✓	✓	✗	6
view	✓	✓	✓	✗	✓	✓	✓	6
log_sqft_lot	✓	✓	✗	✓	✓	✓	✓	6
log_sqft_above	✓	✗	✓	✓	✓	✓	✓	6
floors	✓	✓	✓	✗	✓	✓	✗	5
long	✓	✓	✗	✓	✗	✓	✓	5
house_age	✓	✓	✗	✗	✓	✓	✓	5
log_sqft_lot15	✓	✗	✗	✓	✓	✓	✓	5
condition	✓	✓	✗	✗	✓	✓	✗	4
is_renovated	✓	✓	✗	✗	✓	✓	✗	4
log_sqft_basement	✓	✗	✓	✗	✓	✓	✗	4
waterfront	✗	✓	✗	✗	✓	✓	✗	3
sale_month	✓	✓	✗	✗	✓	✗	✗	3
years_since_renov.	✓	✓	✗	✗	✓	✗	✗	3
has_basement	✓	✓	✗	✗	✗	✗	✗	2

✓	retain
✗	remove

Feature selection was performed by aggregating the results of seven methods, with features retained or removed based on a voting threshold across methods (Seijo-Pardo, Bolón-Canedo and Alonso-Betanzos, 2019). Three candidate feature sets were selected: features selected by at least six methods (V6 with 9 features), features selected by at least four methods (V4 with 16 features), and the top ten features ranked by Random Forest importance scores (RF with 10 features). Each set was assessed across all five models using RMSE to identify the optimal feature combination.

2.4.3 Feature set evaluation

A comparison was conducted across the three candidate feature sets for each model on the training data. Notably, two of the feature sets are nested (V4 and V6), while the third (RF) constitutes a distinct subset derived using different selection criteria. Results shown in Figure 2. Figure 2. Bar chart of feature selection performance analyses.

V4 achieved the lowest RMSE for four of five models, with tree-based models and the stacked ensemble favouring the larger feature set. Linear Regression benefited most from additional features, while Random Forest, Gradient Boosting, and the stacked model performed similarly on RF and V4, indicating robustness; the neural network was the only model to slightly prefer RF. Overall, V4 was selected for subsequent modelling and tuning.

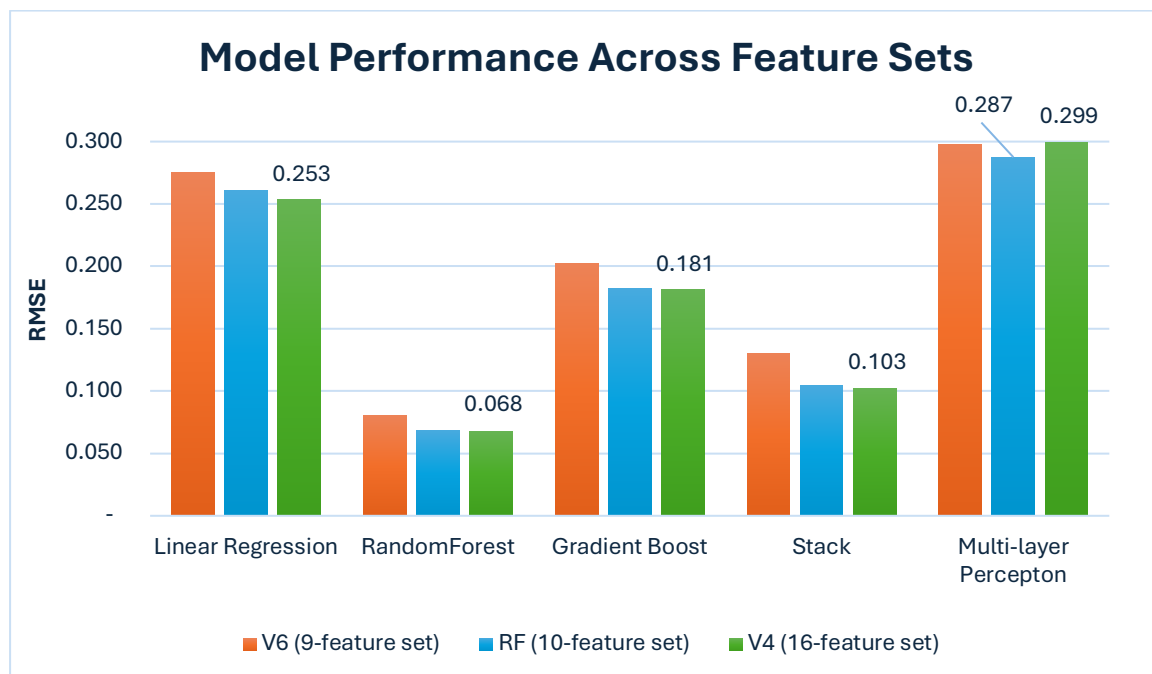


Figure 2. Bar chart of feature selection performance analyses.

2.5 Model Building & Optimisation

2.5.1 Model training

All five candidate models were trained using the V4 feature set. Models were initially fitted using default parameters and on the training partition only, with the test set held back exclusively for final evaluation to ensure an unbiased assessment of performance.

2.5.2 Hyperparameter tuning

Optimal tuning is known to reduce computational overhead while maintaining robust evaluation (Stryker, 2024). Here, each model was tuned using GridSearchCV (scikit-learn, 2026b) with 5-fold cross-validation on the training data only, optimising for RMSE. A hyperparameter grid was defined for each model individually, reflecting the differing parameter spaces across model types. Linear Regression was tuned over fit_intercept; Random Forest over the number of estimators, tree depth, and minimum sample constraints; Gradient Boosting over the number of estimators, learning rate, depth, and subsampling rate; and the Neural Network over hidden layer architecture, activation function, regularisation strength, and learning rate. The best estimators identified for each base model were subsequently used to construct the Stacked Ensemble, using optimally tuned components.

3 Results

3.1 Evaluation

Model performance is reported in three stages: baseline test performance with default parameters (Table 5), the optimal hyperparameters identified via GridSearchCV (Table 6 and Appendix 3), and test performance after applying these tuned settings (Table 7), allowing assessment of the impact of optimisation.

Table 5. Comparative model performance, default hyperparameters.

Model	RMSE	R ²	Adjusted R ²	CV R ² mean	CV R ² std
Stack	0.171	0.895	0.895	0.885	0.004
Random Forest	0.174	0.891	0.891	0.880	0.004
Gradient Boost	0.184	0.879	0.879	0.869	0.006
Linear Regr.	0.255	0.768	0.767	0.767	0.008
MLP	0.302	0.673	0.671	0.679	0.022

Table 6. GridSearchCV-selected hyperparameters (non-default)

Model	Hyperparameters
Linear Regression	-
Random Forest	n_estimators: 500, max_depth: 20, min_samples_split: 5, min_samples_leaf: 1, max_features: 'sqrt,
Gradient Boost	n_estimators: 500, max_depth: 6, learning_rate: 0.05, min_samples_leaf: 50, subsample: 0.8
Neural Networks	hidden_layer_sizes: (100,...), activation: 'tanh', alpha: 0.001

Table 7. Comparative model performance, tuned hyperparameters.

Model	RMSE	R ²	Adjusted R ²	CV R ² mean	CV R ² std
Gradient Boost	0.162	0.906	0.906	0.898	0.005
Stack	0.162	0.906	0.906	0.898	0.005
RandomForest	0.177	0.887	0.887	0.878	0.005
MLP	0.249	0.778	0.777	0.775	0.020
Linear Regr.	0.255	0.768	0.767	0.767	0.008

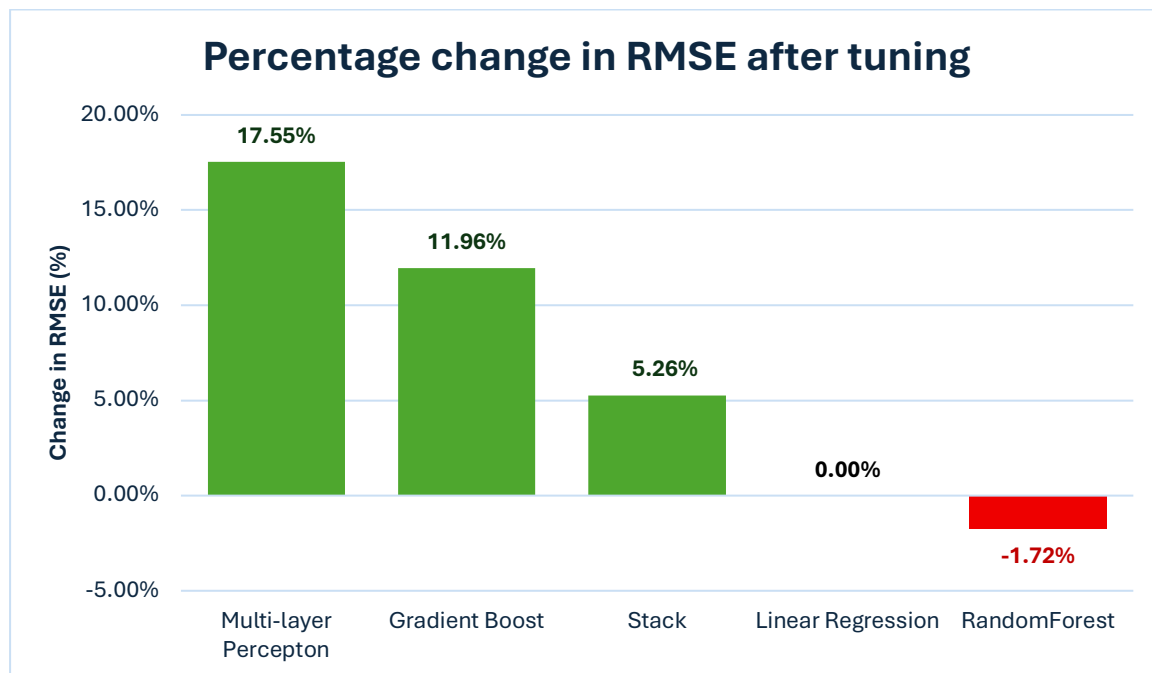


Figure 3. Bar chart of RMSE percentage change after hyperparameter tuning.

In the baseline setting (Table 5), the stacked model achieved the lowest RMSE (0.171), closely followed by Random Forest (0.174) and Gradient Boosting (0.184), whereas Linear Regression and MLP performed substantially worse. After tuning (Table 7), Gradient Boosting and Stacking emerged as the joint best performers (RMSE = 0.162),

with MLP also showing notable improvement (from 0.302 to 0.249), while Random Forest performance slightly declined and Linear Regression remained unchanged. These changes are summarised in Figure 3, which shows the percentage reduction in RMSE relative to the baseline. The largest gain was observed for MLP (17.55%), followed by Gradient Boosting (11.96%) and Stacking (5.26%).

3.2 Comparison

Figure 4 compares model performance on the test set using RMSE and adjusted R^2 . Gradient Boosting and the Stacked model achieved the strongest performance, both recording the lowest RMSE (0.162) and the highest adjusted R^2 (0.906), indicating the best overall fit. Random Forest followed with slightly weaker performance (RMSE = 0.177; adjusted R^2 = 0.887). In contrast, MLP and Linear Regression performed substantially worse, with higher RMSE values (0.249 and 0.255, respectively) and lower adjusted R^2 (0.777 and 0.767). The ranking of models is consistent across both metrics, with tree-based and ensemble approaches outperforming simpler and neural network models in this evaluation.

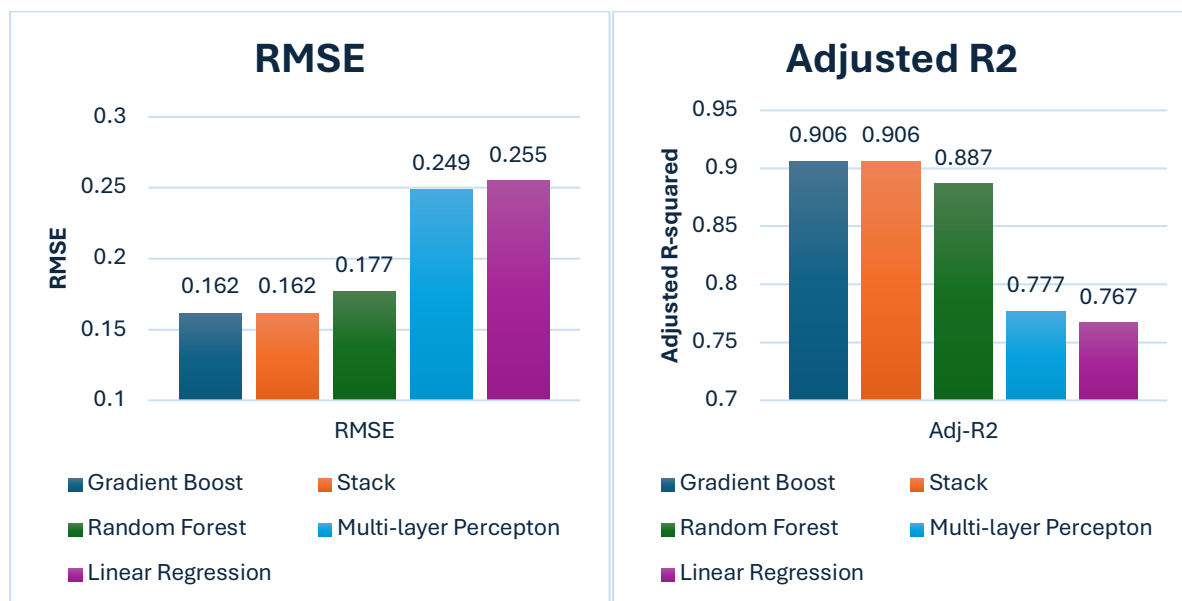


Figure 4. Bar chart of comparative model performance after hyperparameter tuning.

3.3 Validation

Figure 5 shows that across all five cross-validation folds, Gradient Boost (gbr) and Stacking (stack) consistently achieved the lowest RMSE (approximately 0.170), with the gbr line (green) hidden behind the stack line (purple). Random Forest (rf) and linear regression (lr) remained stable but higher, at around 0.180 and 0.25, respectively. The Multi-Layer Perceptron (MLP) showed the greatest instability, peaking sharply at fold 3 (0.30) before recovering to approximately 0.24 by fold 5, suggesting greater sensitivity to data partitioning than the other models.

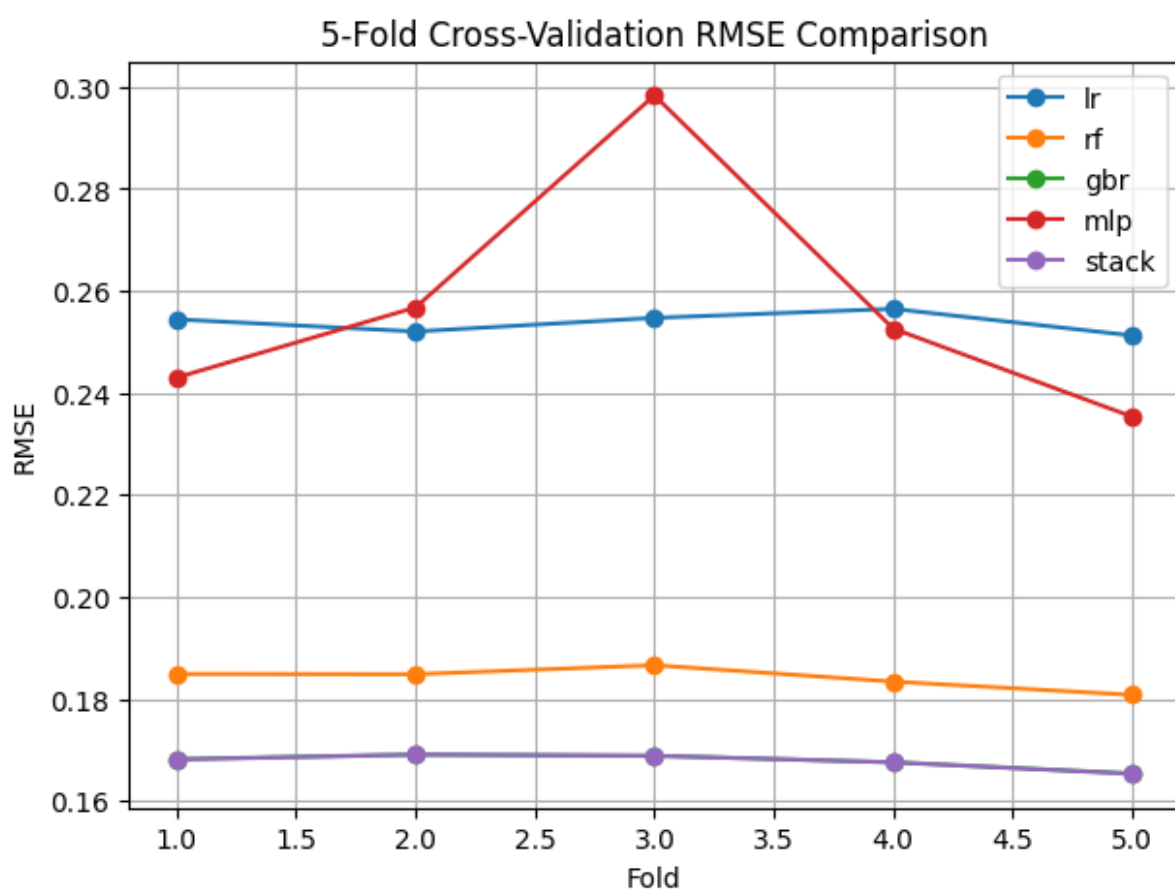


Figure 5. Line plot of 5-fold cross-validation RMSE across models.

Gradient Boost validation

Figure 6 shows the observed versus predicted values for the Gradient Boosting model on the log-transformed price scale. The predicted values align closely with the observed values, with the trend line closely following the perfect prediction line, indicating strong model performance across the price range, though some dispersion is visible at the lower and upper extremes.

Figure 7 shows the distribution of residuals, which are approximately normally distributed and centred around zero, with visible heavy tails.

Figure 8 shows the feature importance scores, indicating that grade and lat are the most influential predictors, each accounting for approximately 0.30 of the total importance, followed by log_sqft_living at approximately 0.20. The remaining features contribute considerably less individually.

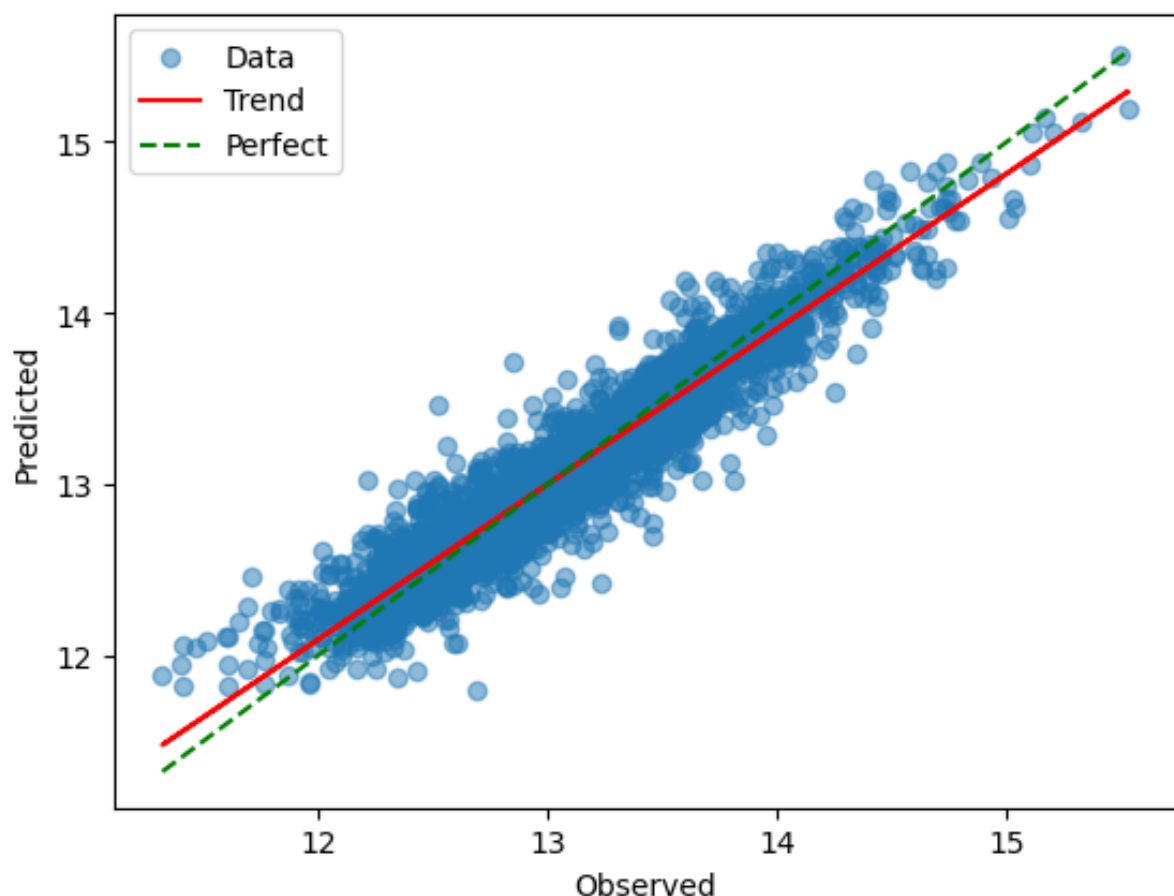


Figure 6. Scatter plot of observed vs predicted values, Gradient Boost.

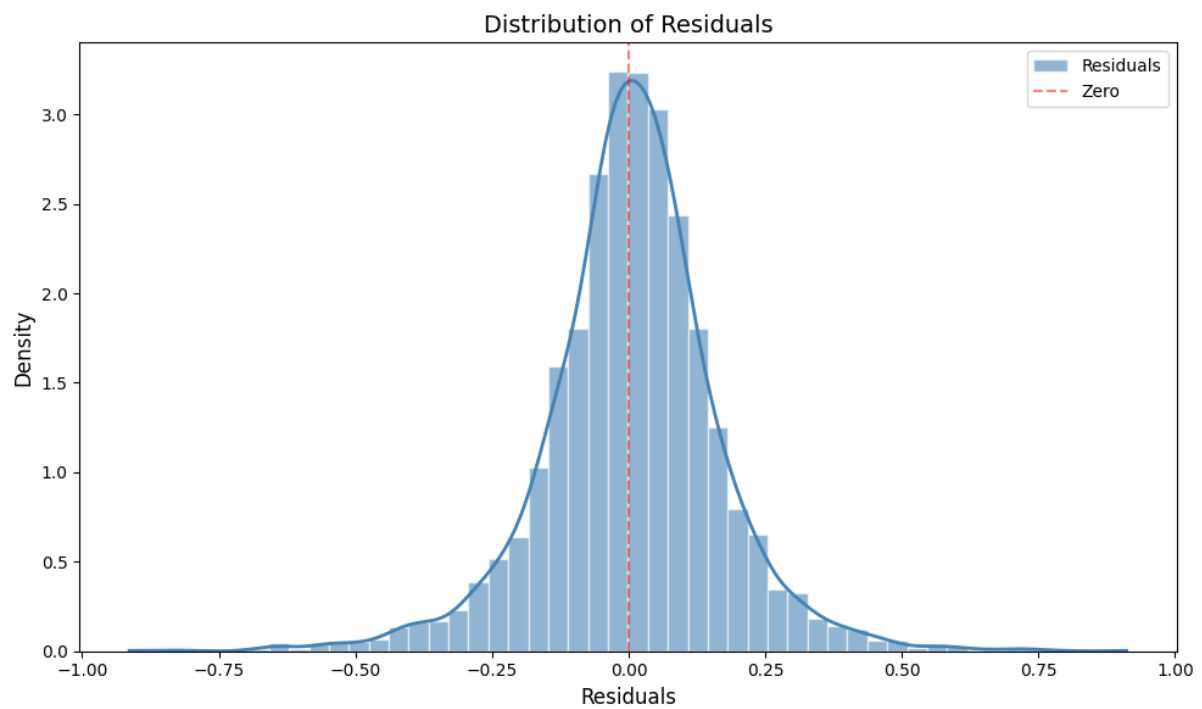


Figure 7. Histogram of residuals, Gradient Boost.

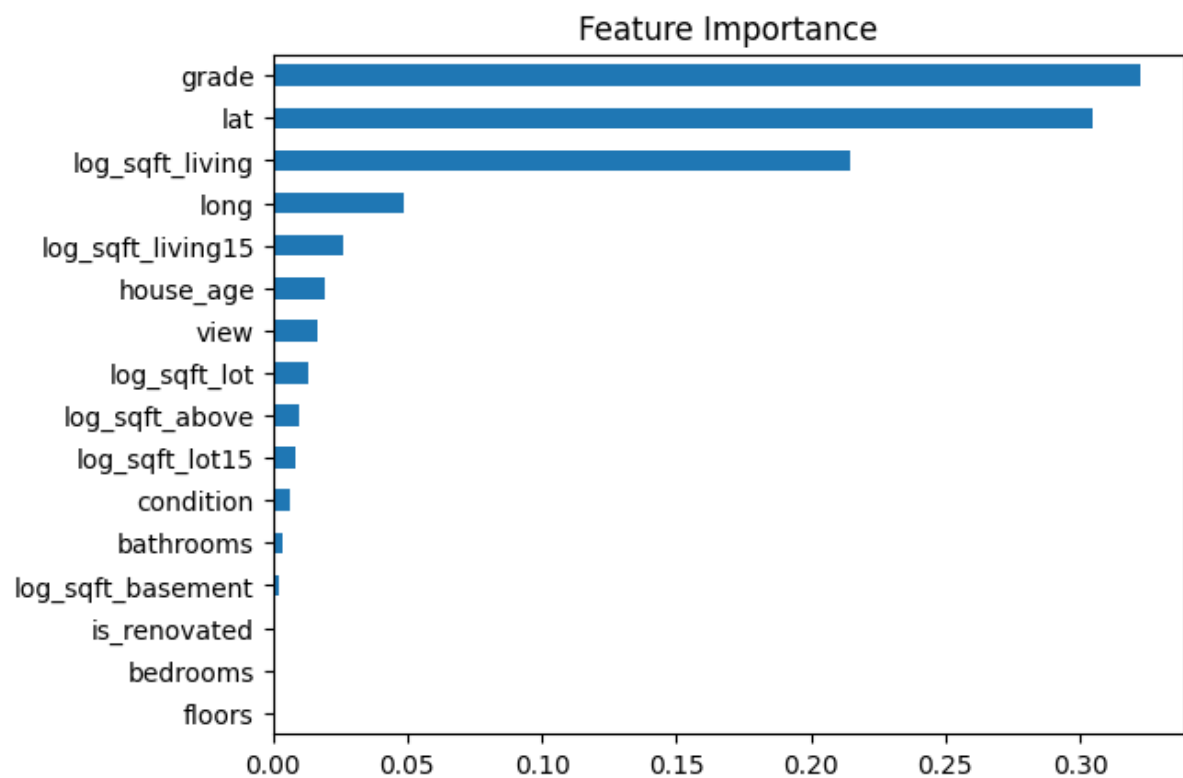


Figure 8. Bar chart of feature importance, Gradient Boost.

4 Discussion

Ensemble methods consistently outperformed both linear regression and the MLP, with gradient boosting and the stacking ensemble emerging as the strongest performers after hyperparameter tuning. Tuning produced mixed outcomes across models — gradient boosting and the MLP both improved meaningfully, while random forest marginally worsened, and linear regression was unaffected, as expected given its constrained functional form.

Because the house price was log-transformed prior to modelling, the RMSE figures require careful interpretation. When predictions were converted back to the original price scale, the Gradient Boosting model yielded an RMSE of \$114,477, representing approximately 21% of the median sale price in the dataset. This is best understood in proportional terms rather than as an absolute error, and it reflects a reasonable level of predictive accuracy given the inherent variance in housing markets.

Feature importance analysis confirmed that property quality (grade), geographic location (latitude), and living space size (sqft_living) were the primary drivers of price prediction, consistent with other studies (Chenxi, 2024; Tang, 2025). The residual distribution was approximately symmetric and centred near zero, indicating no systematic bias. However, slight divergence at higher price points suggests the model is less precise for luxury properties, likely due to their underrepresentation in the training data.

Both the Stacking Ensemble and Gradient Boosting emerged as the strongest and most stable models. Given that Gradient Boosting achieves performance nearly identical to the stacking ensemble whilst being a considerably simpler model, it is the preferred choice — offering equivalent predictive power without the added complexity of stacking.

5 Conclusions

This study evaluated five machine learning models for predicting residential property prices and found that ensemble methods consistently delivered the strongest performance. After hyperparameter tuning, Gradient Boosting achieved an R^2 of 0.906, indicating that the selected features explain most of the variance in house prices. With predictions typically within approximately 21% of the median sale price, the model represents a practical tool for property price estimation.

The findings highlight that model complexity does not guarantee better performance, and that targeted tuning of a well-chosen simpler model can be more rewarding than

relying on a complex architecture. Gradient Boosting responded strongly to hyperparameter optimisation, ultimately matching the performance of the more complex stacking ensemble — suggesting that investing in tuning an appropriate base model is a worthwhile and efficient modelling strategy.

Future work could explore additional feature engineering, the inclusion of other macroeconomic factors, or the application of this approach to other housing markets to assess generalisability.

6 Ethics

The dataset used in this study contains no personally identifiable information and was collected and published for research and educational purposes. No primary data collection, human participants, or sensitive data were involved in this study, and no ethical approval was required.

References

- Aljohani, O. (2022) ‘Developing a stable house price estimator using regression analysis’, *Proceedings of the 5th International Conference on Future Networks and Distributed Systems*. New York, NY, USA: Association for Computing Machinery (ICFNDS ’21), pp. 113–118. Available at: <https://doi.org/10.1145/3508072.3508091>.
- Chenxi, L. (2024) ‘House price prediction using machine learning’, *Proceedings of the 2024 7th International Conference on Signal Processing and Machine Learning* [Preprint].
- Madhuri, CH.R., Anuradha, G. and Pujitha, M.V. (2019) ‘House Price Prediction Using Regression Techniques: A Comparative Study’, *2019 International Conference on Smart Structures and Systems (ICSSS)*. *2019 International Conference on Smart Structures and Systems (ICSSS)*, pp. 1–5. Available at: <https://doi.org/10.1109/ICSSS.2019.8882834>.
- scikit-learn (2026a) *GradientBoostingRegressor*, *scikit-learn*. Available at: <https://scikit-learn/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html> (Accessed: 28 May 2026).
- scikit-learn (2026b) *GridSearchCV*, *scikit-learn*. Available at: https://scikit-learn/stable/modules/generated/sklearn.model_selection.GridSearchCV.html (Accessed: 29 May 2026).
- scikit-learn (2026c) *LinearRegression*. Available at: https://scikit-learn/stable/modules/generated/sklearn.linear_model.LinearRegression.html (Accessed: 26 May 2026).
- scikit-learn (2026d) *MLPRegressor*, *scikit-learn*. Available at: https://scikit-learn/stable/modules/generated/sklearn.neural_network.MLPRegressor.html (Accessed: 28 May 2026).
- scikit-learn (2026e) *RandomForestRegressor*. Available at: <https://scikit-learn/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html> (Accessed: 26 May 2026).
- scikit-learn (2026f) *sklearn.feature_selection*, *scikit-learn*. Available at: https://scikit-learn/stable/api/sklearn.feature_selection.html (Accessed: 28 May 2026).
- scikit-learn (2026g) *StackingRegressor*. Available at: <https://scikit-learn/stable/modules/generated/sklearn.ensemble.StackingRegressor.html> (Accessed: 26 May 2026).

Seijo-Pardo, B., Bolón-Canedo, V. and Alonso-Betanzos, A. (2019) 'On developing an automatic threshold applied to feature selection ensembles', *Information Fusion*, 45, pp. 227–245. Available at: <https://doi.org/10.1016/j.inffus.2018.02.007>.

Sharma, H., Harsora, H. and Ogunleye, B. (2024) 'An Optimal House Price Prediction Algorithm: XGBoost', *arXiv.org* [Preprint]. Available at: <https://doi.org/10.3390/analytics3010003>.

Sidhu, M.K. *et al.* (2025) 'Innovative Predictive Framework for King County Housing Prices: A Comparative Study of Machine Learning and Ensemble Approaches', *2025 7th International Symposium on Advanced Electrical and Communication Technologies (ISAECT)*. *2025 7th International Symposium on Advanced Electrical and Communication Technologies (ISAECT)*, pp. 1–6. Available at: <https://doi.org/10.1109/ISAECT68904.2025.11318703>.

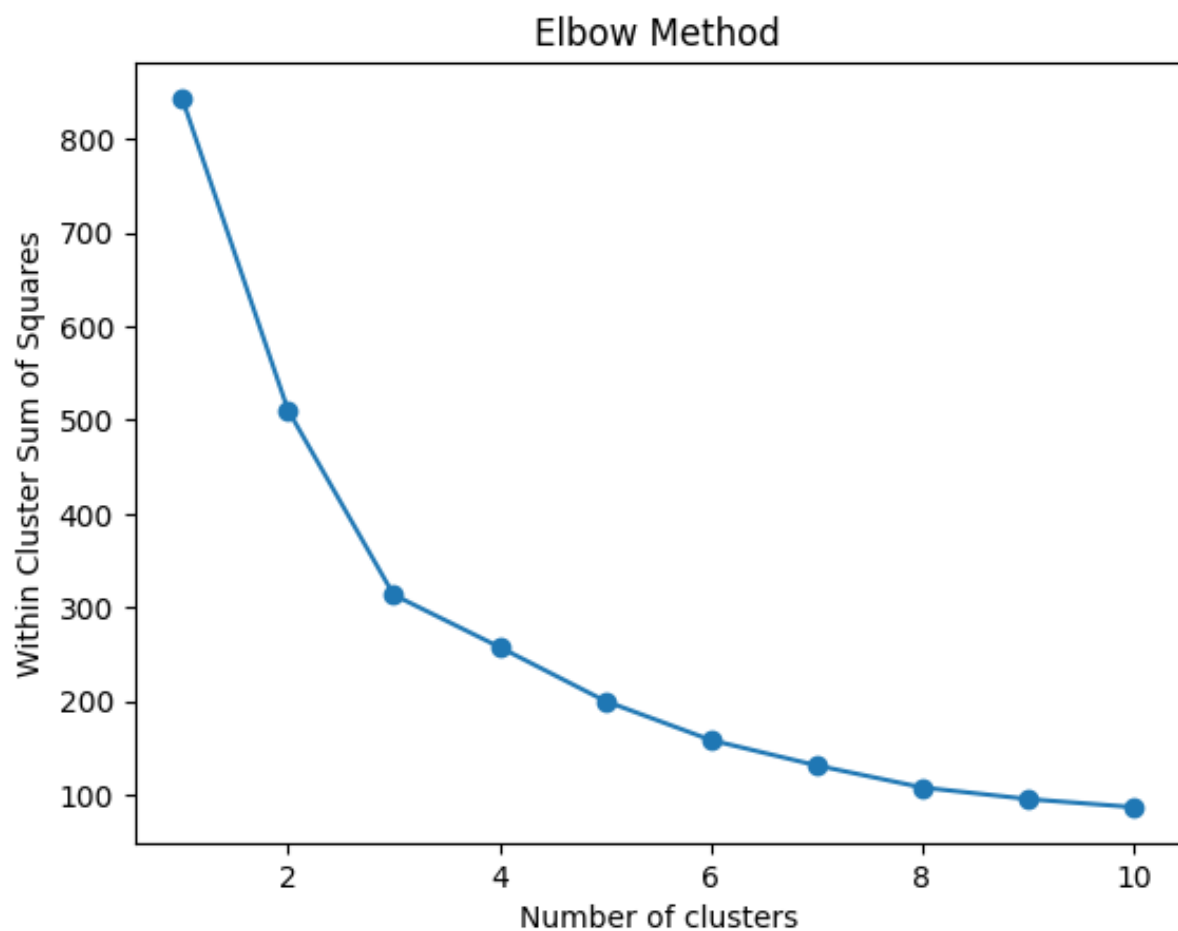
Stryker, I.B., Cole (2024) *What Is Hyperparameter Tuning? | IBM*. Available at: <https://www.ibm.com/think/topics/hyperparameter-tuning> (Accessed: 26 May 2026).

Tang, W. (2025) 'House Price Prediction and Interpretability Analysis Based on Automated Machine Learning', *Proceedings of the 2025 International Symposium on Machine Learning and Social Computing*, pp. 109–114. Available at: <https://doi.org/10.1145/3778450.3778468>.

Appendices

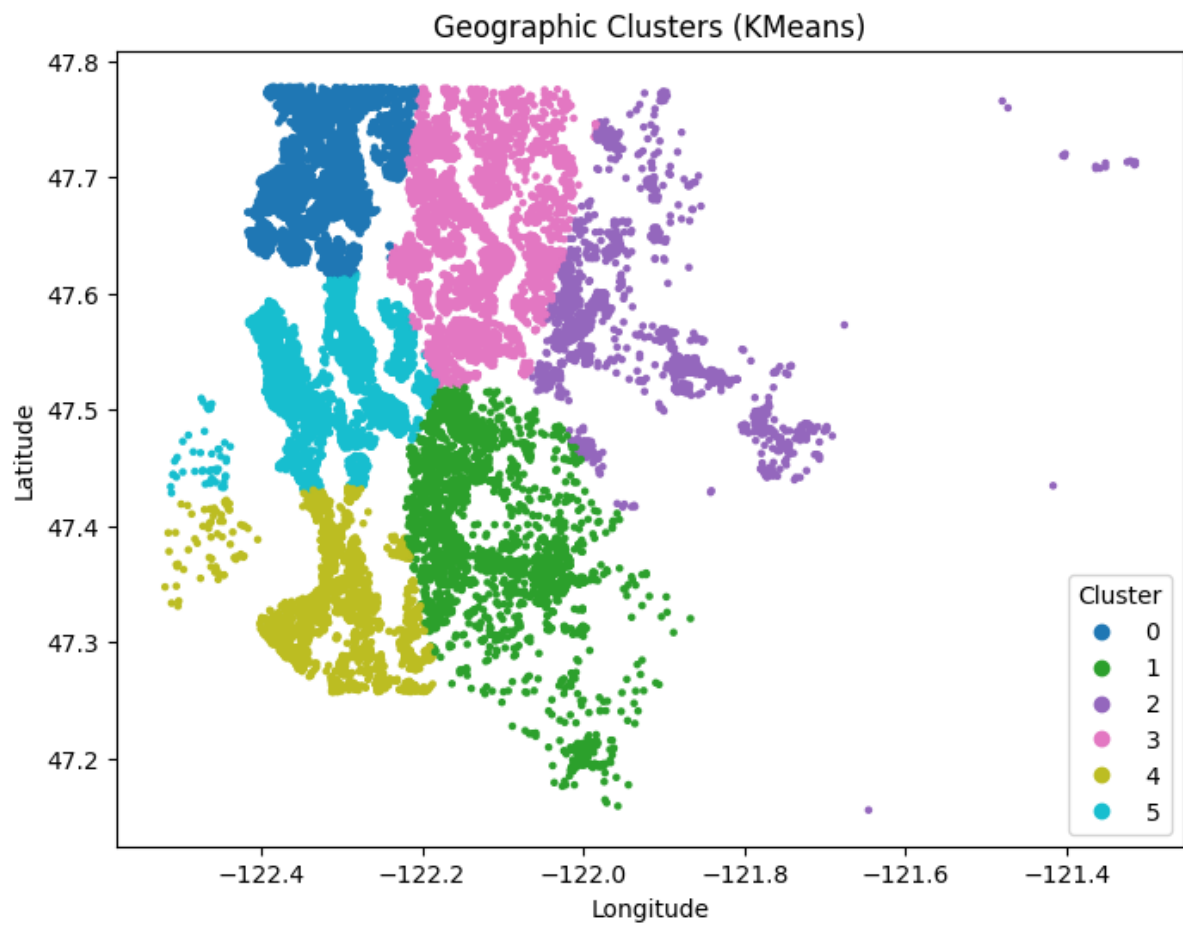
Appendix 2 – Elbow method

Geographical clusters were not included in the main analysis.



Appendix 1. Elbow method for geographical clusters.

Appendix 1 – Geographical clusters



Appendix 2. Map plot of geographical clusters.

Appendix 3 – GridSearchCV-selected parameters

LinearRegression ⓘ ⓘ		
Parameters		
 <code>fit_intercept</code>		True
 <code>copy_X</code>		True
 <code>tol</code>		1e-06
 <code>n_jobs</code>		None
 <code>positive</code>		False

RandomForestRegressor ⓘ ⓘ		
Parameters		
 <code>n_estimators</code>		500
 <code>criterion</code>		'squared_error'
 <code>max_depth</code>		20
 <code>min_samples_split</code>		5
 <code>min_samples_leaf</code>		1
 <code>min_weight_fraction_leaf</code>		0.0
 <code>max_features</code>		'sqrt'
 <code>max_leaf_nodes</code>		None
 <code>min_impurity_decrease</code>		0.0
 <code>bootstrap</code>		True
 <code>oob_score</code>		False
 <code>n_jobs</code>		None
 <code>random_state</code>		42
 <code>verbose</code>		0
 <code>warm_start</code>		False
 <code>ccp_alpha</code>		0.0
 <code>max_samples</code>		None
 <code>monotonic_cst</code>		None

▼ GradientBoostingRegressor ⓘ ⓘ		
▼ Parameters		
	<code>loss</code>	'squared_error'
	<code>learning_rate</code>	0.05
	<code>n_estimators</code>	500
	<code>subsample</code>	0.8
	<code>criterion</code>	'friedman_mse'
	<code>min_samples_split</code>	2
	<code>min_samples_leaf</code>	50
	<code>min_weight_fraction_leaf</code>	0.0
	<code>max_depth</code>	6
	<code>min_impurity_decrease</code>	0.0
	<code>init</code>	None
	<code>random_state</code>	42
	<code>max_features</code>	None
	<code>alpha</code>	0.9
	<code>verbose</code>	0
	<code>max_leaf_nodes</code>	None
	<code>warm_start</code>	False
	<code>validation_fraction</code>	0.1
	<code>n_iter_no_change</code>	None
	<code>tol</code>	0.0001
	<code>ccp_alpha</code>	0.0

MLPRegressor ⓘ ⓘ		
Parameters		
	<code>loss</code>	'squared_error'
	<code>hidden_layer_sizes</code>	(100, ...)
	<code>activation</code>	'tanh'
	<code>solver</code>	'adam'
	<code>alpha</code>	0.001
	<code>batch_size</code>	'auto'
	<code>learning_rate</code>	'constant'
	<code>learning_rate_init</code>	0.001
	<code>power_t</code>	0.5
	<code>max_iter</code>	200
	<code>shuffle</code>	True
	<code>random_state</code>	42
	<code>tol</code>	0.0001
	<code>verbose</code>	False
	<code>warm_start</code>	False
	<code>momentum</code>	0.9
	<code>nesterovs_momentum</code>	True
	<code>early_stopping</code>	False
	<code>validation_fraction</code>	0.1
	<code>beta_1</code>	0.9
	<code>beta_2</code>	0.999
	<code>epsilon</code>	1e-08
	<code>n_iter_no_change</code>	10
	<code>max_fun</code>	15000

Appendix 3. GridSearchCV-selected parameter

